

Sprint 1 Week 3 — CLI Prototype Requirements

1. Purpose

The CLI prototype will provide developers with a simple command line method for testing the blockchain data collection and scam detection pipeline before it is connected to the final chat interface.

The CLI should allow a developer to provide a supported blockchain input, determine what type of input has been supplied, run only the relevant available tools, display the outcome of each check, and save the results using the agreed JSON output structure.

The CLI is a testing prototype only and is not intended to represent the final user experience.

2. CLI Entry Screen

When started, the CLI should display:

- Application/project name.
- A short description or identifier indicating that it is the blockchain security/scam-detection prototype.
- A clearly labelled input prompt.

Example:

```
ChainGuard  
Blockchain Security Assistant
```

```
Enter wallet, token address or transaction hash:
```

```
>
```

3. Supported User Inputs

For the Week 3 prototype, the CLI should support the blockchain inputs currently required for testing the implemented fetchers.

3.1 EVM Wallet Address

Example:

```
0x00192fb10df37c9fb26829eb2cc623cd1bf599e8
```

The CLI should identify the input as an EVM compatible address and determine whether it represents a wallet or contract where possible.

3.2 EVM Token / Contract Address

Example:

0x6C1e7FfAe984b5644C2ab95FC3aDF5794317C6aE

The CLI should allow the prototype to identify contract/token related inputs so appropriate token specific checks can be used.

3.3 Transaction Hash

Example:

0xbc78ab8a9e9a0bca7d0321a27b2c03addeae08ba81ea98b03cd3dd237eabed44

The CLI should recognise a transaction hash separately from a wallet or contract address.

Where available, the transaction resolver should return the addresses associated with the transaction.

4. Input Classification

After receiving an input, the CLI should clearly identify the detected input information.

The prototype should distinguish, where possible, between:

- EVM wallet.
- EVM delegated wallet.
- EVM contract.
- EVM token contract.
- Transaction hash.
- Unknown / unsupported input.

Example CLI output:

Input recognised.

Chain family: EVM

Address type: wallet

If the type cannot be determined confidently, the CLI should display:

Address type: unknown

rather than making an unsupported assumption.

5. Appropriate Tool Selection

The CLI should not attempt every available API or detector against every input.

Only relevant tools should be used based on the identified input type.

For example:

EVM Wallet

Potentially relevant:

- Transaction history.
- Token information.
- Contract/bytecode check where needed for classification.

Token only tools should be skipped.

EVM Token Contract

Potentially relevant:

- Contract information.
- Transaction information.
- Honeypot check.
- Liquidity/pool information.

Transaction Hash

Potentially relevant:

- Transaction hash resolver.

Address or token-specific checks should not be run directly against the transaction hash.

6. Detection / Fetcher Tool Output

Every tool called through the CLI should follow a consistent wrapper structure.

Each tool result must contain:

```
{  
  "status": "pass",  
  "data": {},  
  "error": null  
}
```

The allowed status values are:

pass

The tool completed successfully and returned usable information.

```
{
  "status": "pass",
  "data": {},
  "error": null
}
```

fail

The tool was applicable and attempted, but an error occurred.

```
{
  "status": "fail",
  "data": null,
  "error": "Unable to retrieve information from API."
}
```

skip

The tool was not appropriate for the supplied input.

```
{
  "status": "skip",
  "data": null,
  "error": "This tool supports EVM token contracts only."
}
```

A skipped tool is not considered a failure.

7. Detection Tools

The CLI should support the detection/data tools implemented by the development team.

These currently include areas such as:

- Contract information.
- Transaction history.
- Token information.
- Liquidity / paired pool information.
- Transaction hash resolution.
- Honeypot checking.
- RugCheck.

The purpose of the CLI is to coordinate and test these tools rather than duplicate their internal logic.

The client specification requires the eventual application to use built in fetchers for contract, transaction, token and liquidity information.

8. Scam Detection Tool Output

Where a scam-detection tool returns an actual assessment, the output should be structured so it can later be converted into the agreed scam detection format.

The eventual standard detection result should be capable of representing:

```
{
  "label": "scam",
  "risk_type": "rug_pull",
  "confidence": 0.82,
  "evidence": [
    {
      "description": "Liquidity was rapidly removed.",
      "weight": 0.7
    }
  ]
}
```

The standard assessment labels should support:

- scam
- not_scam
- insufficient_evidence

However, **the Week 3 CLI does not need to generate its own scam conclusion if the current tools do not yet provide one.**

At this stage, it is acceptable for the CLI to collect and standardise the evidence that will later be consumed by an external detector or the LLM Scam Checker.

The client's final DetectionResult requires a label, risk type, confidence and evidence.

9. CLI Status Output

After processing the input, the CLI should show a readable status for each tool.

Example:

Input type: EVM / token_contract

contract: PASS
transactions: PASS
honeypot: PASS
liquidity: PASS
rugcheck: SKIP
token_info: SKIP

10. Error Handling

The CLI should handle errors without terminating the entire analysis where possible.

Examples include:

- Invalid input.
- Missing API key.
- API timeout.
- External service unavailable.
- Unsupported blockchain.
- No transaction found.
- Tool not applicable to the input.
- No data returned.

If one tool fails, other applicable tools should still be allowed to complete.

Example:

contract: PASS
transactions: FAIL
Unable to contact Etherscan
liquidity: PASS

The CLI should not report a tool as successfully completed when it failed or was skipped.

11. Invalid Input

If the value does not match a supported input format, the CLI should clearly tell the developer.

Example:

Input could not be recognised.

Supported inputs:

- EVM wallet/contract address
- Transaction hash
- Supported Solana token address

The system should not silently guess the input type.

12. Combined JSON Output

After processing is complete, the CLI should save one combined JSON result using the structure defined in the team's **JSON Output Requirements**.

At minimum the combined result should contain:

```
{
  "input": {},
  "address_analysis": {},
  "api_checks": {},
  "risk_evaluation": {},
  "summary": {}
}
```

For Week 3, risk_evaluation may be empty or null if no complete scam assessment has yet been performed.

13. Summary Output

The CLI should provide a simple processing summary.

Example:

Checks complete.

Passed: 4

Failed: 1

Skipped: 2

Combined results saved to:
fetchers/json_files/info.json

The JSON should also contain these counts.

14. Example CLI Flow — Wallet

ChainGuard

Blockchain Security Assistant

Enter wallet, token address or transaction hash:

> 0x00192fb10df37c9fb26829eb2cc623cd1bf599e8

Input recognised.

Chain family: EVM

Address type: wallet

Running checks...

contract: PASS
transactions: PASS
token_info: PASS
honeypot: SKIP
liquidity: SKIP
rugcheck: SKIP

Checks complete.

Passed: 3

Failed: 0

Skipped: 3

Combined results saved.

15. Example CLI Flow — Transaction Hash

Enter wallet, token address or transaction hash:

> 0xbc78ab8a9e9a0bca7d0321a27b2c03addeae08ba81ea98b03cd3dd237eabed44

Input recognised.

Input type: transaction_hash

tx_hash: PASS
contract: SKIP
transactions: SKIP
honeypot: SKIP
liquidity: SKIP
rugcheck: SKIP
token_info: SKIP

Transaction addresses identified:

From: 0x00192f...

To: 0xc67f4e...

Combined results saved.

16. Out of Scope for Week 3 CLI

The following are not required for this CLI prototype:

- Final web interface.
- ChatGPT style conversational UI.
- User accounts or login.
- Voice input.
- Full natural language interpretation.
- Network graph visualisation.
- Training a scam detection model.
- Production deployment.
- Polished user facing design.
- Full LLM conversation handling.

The CLI exists primarily so developers can test the underlying pipeline and agree on how information flows between components.

17. Acceptance Criteria

The Sprint 1 Week 3 CLI prototype will satisfy the requirements when:

- A developer can launch the CLI locally.
- The CLI provides a clear input prompt.
- Supported EVM addresses can be entered.
- Transaction hashes can be recognised.
- Supported Solana token addresses can be recognised where applicable.
- Input type/address type is displayed.
- Only applicable tools are run.
- Non applicable tools are marked as skip.
- Successful tools are marked as pass.
- Failed tools are marked as fail with an error message.
- One failed tool does not prevent other tools from returning results.
- Results use the agreed JSON structure.
- A combined JSON file is produced.
- A summary of passed, failed and skipped checks is displayed.
- The prototype does not require the final web/chat interface.